

StriimWatcher: Monitoring Source OP

Many customers want an ability to retrieve monitoring data without needing to create separate apps (such as using Python or curl to run API commands). Instead, some customers want to periodically retrieve various this data in table structure. Striim Watcher is a custom Source OP, which can be used to get this data.

Overview

What is StriimWatcher?

StriimWatcher is a Field-Developed component that takes monitoring metrics, and generates WAEvents in the form of tables. It is a “Source” component.

Notes for Upgrading (DDL Changes!):

Check the [BQ](#) or the [PostgreSQL](#) GitHub SubFolder Upgrade to add columns to the changed tables:

- striim_mon_datawarehouse_detail: new column
- striim_mon_table_comparison: 2 new columns
- striim_mon_table_comparison_sli: 2 new columns

How do I install StriimWatcher?

Step 1: Download the appropriate version.

Find the latest download in [GitHub](https://github.com/daniel-striim/StriimWatcher): <https://github.com/daniel-striim/StriimWatcher>

The version of StriimWatcher should match the version of Striim you are running in. This document has been updated to the latest version 5.2.5, which includes new tables and new columns in existing tables.

(Optional) Step 1.5: Unload existing Open Processor

In Striim Console in the UI, when you run `list libraries`; if you see StriimWatcher, then you need to UNLOAD it.

```
-- Processing - list libraries
```

*	File Name	

1	StriimWatcher-5.2.5.jar	

```
-- -> SUCCESS
```

```
-- Elapsed time: 41 ms
```

You will run the unload command referencing the listed jar filename; note it *must* still exist in your UploadedFiles in order to UNLOAD it.

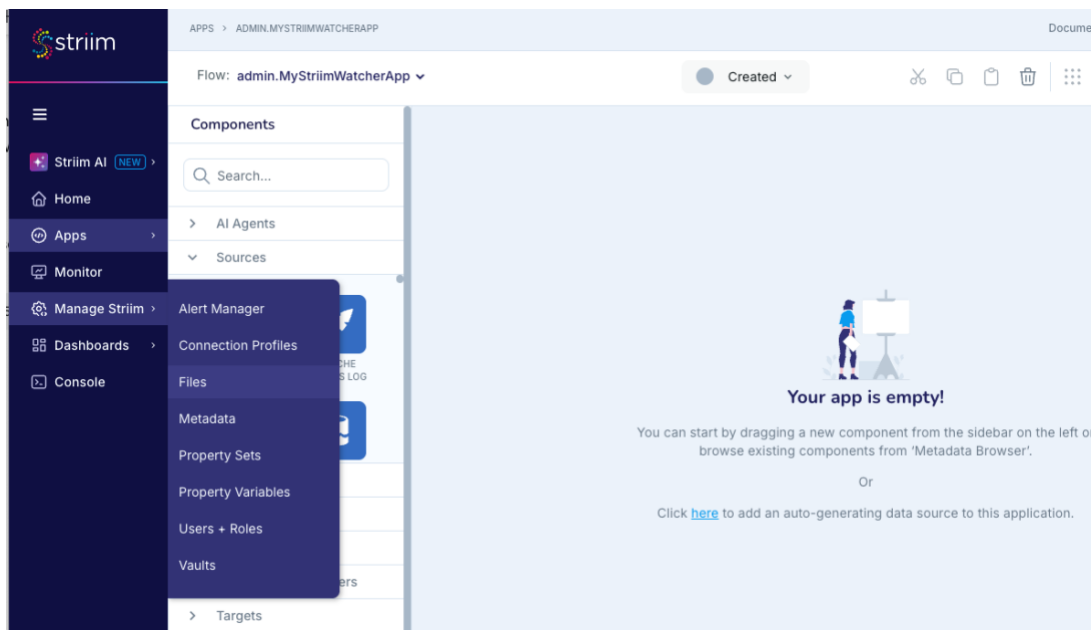
For example, if the above File Name listed StriimWatcher-5.2.4.jar, you would unload it like so:

```
UNLOAD OPEN PROCESSOR 'UploadedFiles/StriimWatcher-5.2.4.jar';
```

Step 2: Upload the Open Processor via the UI

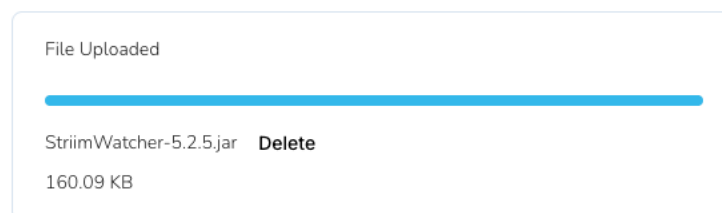
You will need to upload the provided StriimWatcher*.jar file to your Files in the UI.

Do not load it on the Striim server backend, as this may not provide the right file ownership and/or permissions. Do not copy it to the lib or modules directory, as this may cause issues unloading or replacing in the future.



Upload the latest StriimWatcher.jar file on the 'Files' Page:

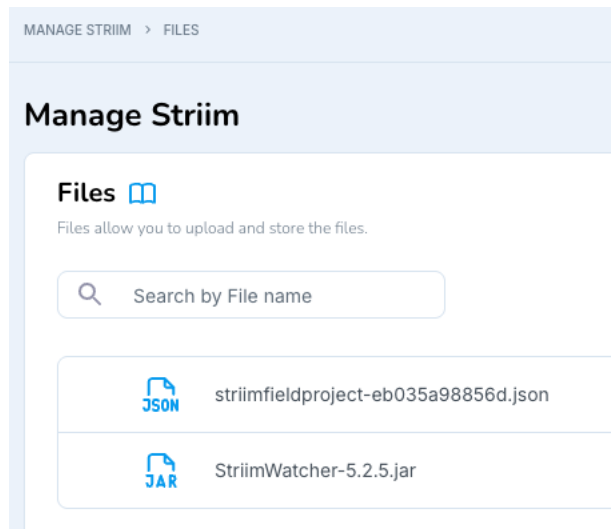
Upload File



Cancel

Done

It should now appear in your files list:



Step 3: Run the LOAD command in console

In Console, run the 'Load' command for StriimWatcher:

```
LOAD OPEN PROCESSOR 'UploadedFiles/StriimWatcher-5.2.5.jar';
```

You should see a SUCCESS. Note that `list libraries;` will now show StriimWatcher is loaded:

```
-- Processing - use admin
-- -> SUCCESS
-- Elapsed time: 28 ms
-- Processing - LOAD OPEN PROCESSOR 'UploadedFiles/StriimWatcher-5.2.5.jar'
-- -> SUCCESS
-- Elapsed time: 114 ms
-- Processing - list libraries

+-----+-----+
| * |      File Name      |
+-----+-----+
| 1 | StriimWatcher-5.2.5.jar |
+-----+-----+

-- -> SUCCESS
-- Elapsed time: 26 ms
```

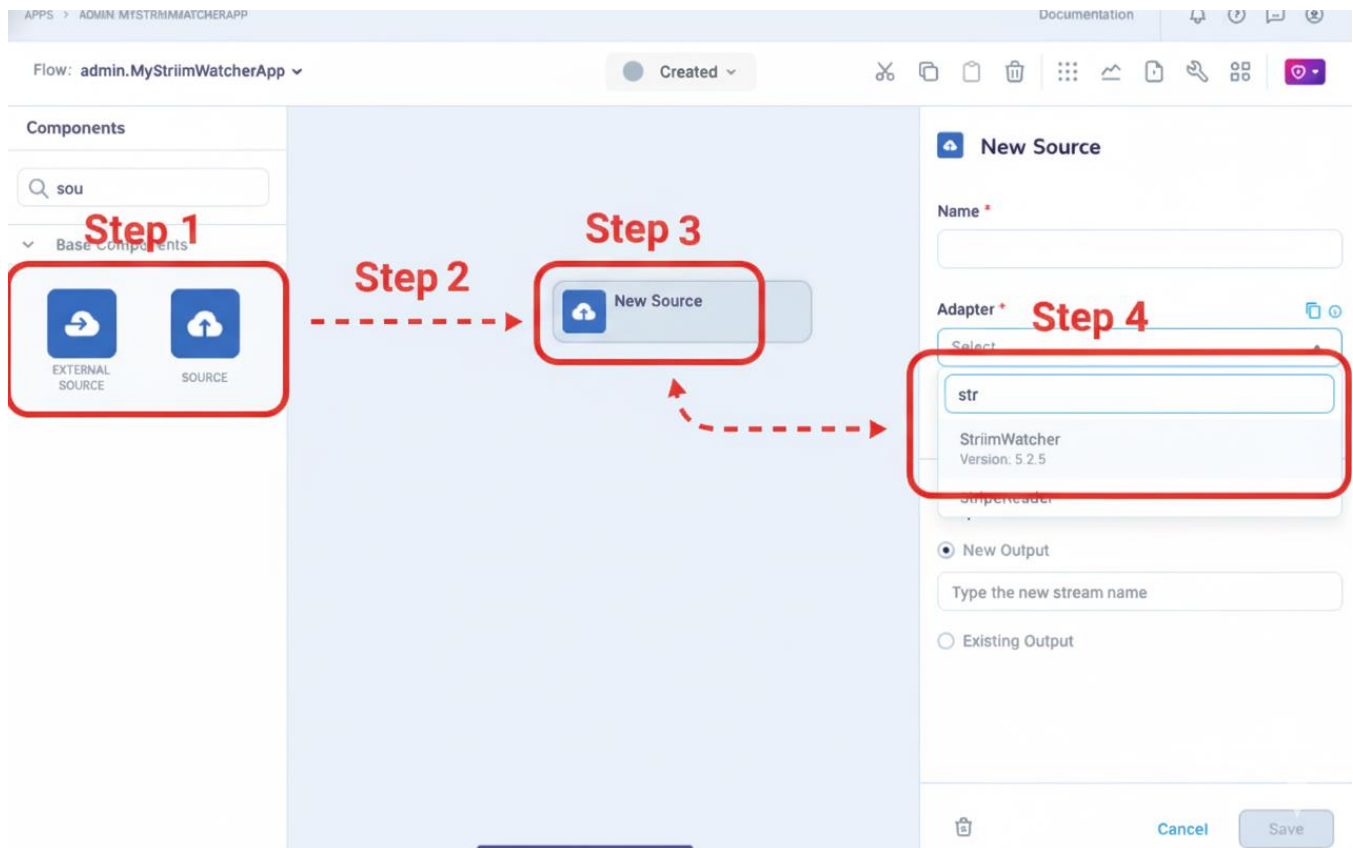
Step 4: Set up the DDL for your target

Since this is a source, you can send this information anywhere. Please find the appropriate download links above. If your target is missing, please reach out to Field Engineering to assist with creation.

DDL should be available in GitHub at: <https://github.com/daniel-striim/StriimWatcher/tree/main/DDL>

Step 5: Create a New App with a 'Source' Component

Create a new app, and search the literal word 'source' on the left of the screen. Drag that generic 'source' object to the canvas. Then, select StriimWatcher as the 'Adapter' type.



Note: If “StriimWatcher” does not appear in your list after searching in Step 4, do a refresh of your browser page and try again.

Step 6: Configure your options

This guide explains what each option does. Examine them to determine which DDL objects will be populated, and understand which data will populate it.

Sample App TQL (StriimWatcher Defaults):

```
CREATE APPLICATION MyStriimWatcherApp;

CREATE OR REPLACE SOURCE MySWSSource USING Global.StriimWatcher VERSION '5.2.5' (
  RepeatInSeconds: '300',
  StartOn: '2020-12-31T08:00',
  PreserveStriimWatcherPosition: false,
  IncludeLogWatcher: false,
  IncludeDebugLogWithLogWatcher: false,
  IncludeCommandLogWithLogWatcher: false,
  IncludeSystemCommandsWithLogWatcher: false,
  AutohealMonitoringApps: false,
  StriimAPIHttpProtocol: 'http',
  StriimAPIHttpHostname: 'localhost',
  StriimAPIHttpPort: '9080',
  IncludeNodeMonitor: true,
  IncludeNodeCluster: true,
  IncludeNodeES: true,
  IncludeAppDetail: true,
  IncludeAppDescribeDetail: true,
  IncludeAppStatusDetail: true,
  IncludeCreatedApplicationDetail: false,
  IncludeDeployedApplicationDetail: false,
  IncludeLee: true,
  IncludeTableComparisonDetail: true,
  IncludeTableComparisonDetail_SinceLastInterval: true,
  IncludeDataWarehouseDetails: false,
  IncludeTargetInformation: true,
  IncludeTargetInformationDetail: false,
  IncludeSourceInformation: true,
  IncludeSourceInformationDetail: false,
  IncludeSystemConfiguration: true,
  IncludeSystemConfigurationDetail: false,
  IncludeOnlyNoticedConfChanges: false,
  IncludeCheckpointHistoryDetail: false,
  IncludeOracleOpenTrx: false,
  IncludeFileLineage: false,
  IncludeComponentDetailsAsOutput: false,
  IncludeTypeDetailsAsOutput: false )
OUTPUT TO MySWS_Output;

END APPLICATION MyStriimWatcherApp;
```

Note: You *must* export, drop, and unload StriimWatcher between major upgrades. Reach out to your Field Engineer to provide you with the correct version for your version of Striim.

Debugging:

This is a field component, so occasionally we will need debugging information to investigate root cause of issues. If needed, this explains how to enable and disable debugging logging.

Enable Debugging: Run the following commands in Striim Console UI:

Note: this will save on the server to *logs/striim.server.clidebug.log*:

```
set loglevel = {com.striim.util.StriimWatcher: debug};
```

Run the following to disable:

Note: Be sure to the logging back to info once you have collected data (~10 minutes, as long as the 'interval' you have selected to run StriimWatcher has passed.

```
set loglevel = {com.striim.util.StriimWatcher: info};
```

Performance

In testing, a StriimWatcher has been optimized to utilize minimal resources (with default settings). However, depending on the options selected, some data is retained in memory. Like any app, monitor its usage with the options selected, and work with your Field Engineer to understand which options may impact performance.

Parameters

1. Run-time Parameters

1. Runtime

Repeat In Seconds *

300

ADV

HIDE ADVANCED SETTINGS ^

Start On

2020-12-31T08:00

End On

☒

Preserve Striim Watcher Position

☒

Include Log Watcher

☒

Include Debug Log With Log Watcher

☒

Include Command Log With Log Watcher

☐

Include System Commands With Log Watcher

☒

Autoheal Monitoring Apps

☐

Auto Vault Checkin

Striim API Http Protocol

http

Striim API Http Hostname

localhost

Striim API Http Port

9080

Repeat in Seconds:

- The number of seconds between each run (default 300 seconds → 5 minutes).
- At each interval, generates a table entry: `mon.striim_mon_table_runhistory`.
- **Note:** `mon.striim_mon_table_runhistory` keeps track of how long the run took (stored in field `runtimeDurationMS`); ensure your Repeat in Seconds setting is longer than this period. It is not recommended to run this more often than every 5 minutes.

Start On (Not Required/Uncommon)

- A datetime that reflects when the app can start. Using a historical date will mean it runs. Example:
`2023-11-10T1:20:00`

End On (Not Required/Uncommon)

- A datetime that reflects when the app should stop. It should normally be left blank to run indefinitely.

Preserve Striim Watcher Position:

If true, StriimWatcher will attempt to save the historical table row counts, as well as the last run time. Upon restart of StriimWatcher where this is true, it will re-load the data, and continue from that point.

- Note that this attempts to save these files under the base Striim installation directory:
 - 'StriimWatcher.hist' for table count history.
 - 'StriimWatcher_log.hist' for log watcher position history.
 - Other *.hist tables depending on options chosen.

Include Log Watcher

(On-Prem Only - **Not Available on StriimCloud**):

This attempts to look at the `striim.server.log`, look for errors/smart alerts, and report those as an output event. The position is saved while the app is running (if Preserve Striim Watcher Position flag is utilized, it will save across restarts of the app). Two items from `striim.server.log` will be tracked:

- ERROR: If an error is found, it will report the error message, as well as 20 context lines in either direction.
- SmartAlerts: If a SmartAlert is found, it will attempt to detect the app and report that in `appName`.
- Generates the following table entry: `mon.striim_mon_log_watcher`.

Include Debug Log With Log Watcher

(On-Prem Only - Not Available on StriimCloud):

If enabled, it parses out the Striim debug log `striim.server.clidebug.log` and puts entries in the Log Watcher table: `mon.striim_mon_log_watcher`.

Include Command Log With Log Watcher

(On-Prem Only - Not Available on StriimCloud):

If enabled, it parses out the Striim debug log `striim.command.log` and puts entries in the Log Watcher table: `mon.striim_mon_log_watcher`.

Include System Commands With Log Watcher

(On-Prem Only - Not Available on StriimCloud):

Many commands, including those executed by StriimWatcher, do not have a specific user attached (can often be NULL). By default, events for these commands are not included in the output. If you want to include these entries, then enable this.

Autoheal Monitoring Apps

Striim has various monitoring apps running on Striim nodes. Various conditions can cause these apps to enter a `TERMINATED` state, interrupting the collection of metrics; this can also impact the availability of StriimWatcher data.

If true, this will attempt to run the following review and auto-heal commands (status <node>):

1. Gather Status:

```
status S10_0_0_4;
```

These system monitoring apps can be terminated:

```
-- Processing - status S10_0_0_4

Deployed Applications in Server: S10_0_0_4
  Global.MonitoringProcessApp: RUNNING          <--- Monitored/Auto-Resumed
  Global.MonitoringSourceApp: RUNNING          <--- Monitored/Auto-Resumed
  System$Alerts.AlertingApp: RUNNING           <--- Monitored/Auto-Resumed
  System$Notification.NotificationSourceApp: TERMINATED <--- Monitored/Auto-Resumed
  System$PIIGlobal.SystemAIGlobalApp: RUNNING  <--- Monitored/Auto-Resumed
  Global.adhocquery1f0c0b362924f019a124a675be163bf: CREATED
  admin.SQLCDC: RUNNING
  admin.TEstPGApp: RUNNING
Total: 8, CREATED: 1, RUNNING: 6, TERMINATED: 1
Application Manager Running: Yes

-- -> SUCCESS
-- Elapsed time: 147 ms
```

2. If any are noticed as terminated, the following command is issued (resume appName):

```
RESUME System$Notification.NotificationSourceApp;
```

This will log the following messages as INFO messages in `striim.server.log`:

```
AutohealMonitoringApps: Resuming System$Notification.NotificationSourceApp on node
S10_0_0_4 (status was: TERMINATED)
```

```
AutohealMonitoringApps: Successfully resumed
System$Notification.NotificationSourceApp
```

This option does not generate an output.

Auto Vault Checkin

If this option is selected, it will run the following command to 'checkin' on the vaults:

```
list vaults;
```

And then run:

```
read all from <vaultName>;
```

This is utilized to trigger auto-renew connectivity/auth token in instances where there may be quiet periods otherwise without Vault utilization.

This option does not generate an output.

Connectivity Parameters

There are various parameters that are related to where the information is gathered. In instances where you are running StriimWatcher on a Striim Agent, you may need to adjust some of these parameters.

If you are deploying on a Striim Node, then leave these parameters as default. ***Only change the following Connectivity Parameters if StriimWatcher has an issue collecting data or otherwise advised by a Field Engineer.***

Striim API Http Protocol (Not Required/Uncommon)

- Default: `http`
- Optional: `https`
 - If changing the API to `https`, make sure it fits the configured parameters in `startUp.properties` and adjust Striim API Http Port as well.

Striim API Http Hostname (Not Required/Uncommon)

- Default: `localhost`
- Optional: ***IP Address*** or ***DNS Name*** for Striim Node

Striim API Http Port (Not Required/Uncommon)

- Default: `9080`
 - Optional: `9081` (or as assigned via `startUp.properties`)
 - Only override if HTTP/port 9080 has been disabled on your servers.
-

2. NodeDetails Parameters

2. NodeDetails

- ☒ Include Node Monitor *
- ☒ Include Node Cluster *
- ☒ Include Node ES *

Include Node Monitor

- Default: True
- When true, generates a table entry at each interval with details of apps from the `mon;` command:
`mon.striim_mon_node_applications`

Striim Applications						
Name	Status	Rate	SourceRate	CPU%	Nodes	Activity
admin.MyStriimWatcherApp	CREATED	0	0		0	
admin.TestPGApp	RUNNING	0	0	0.029%	1	2025-11-13 09:44:49
admin.SQLCDC	RUNNING	0	0	0.287%	1	2025-11-13 09:44:49

Include Node Cluster

- Default: True
- When true, generates a table entry at each interval with details of Cluster Nodes from the `mon;` command:
`mon.striim_mon_node_cluster`

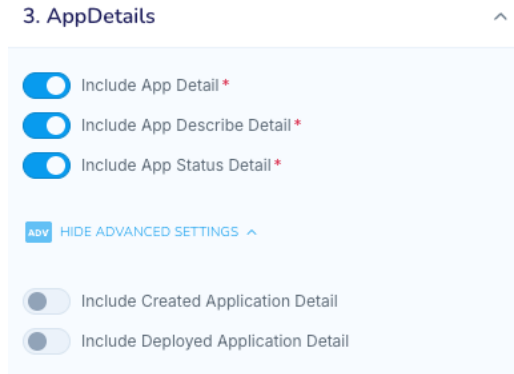
Striim Cluster Nodes				
Name	Version	Free Mem	CPU%	Uptime
S10_0_0_4	5.2.0.4 (4b9b8856d8)	3.58Gb	3%	02H:06M:26S

Include Node ES

- Default: True
- When true, generates a table entry at each interval with details of Cluster Nodes from the `mon;` command:
`mon.striim_mon_node_elasticsearch`

ElasticSearch	
Elasticsearch Receive Throughput	0
Elasticsearch Transmit Throughput	0
Elasticsearch Cluster Storage Free	24,413,097,984
Elasticsearch Cluster Storage Total	494,384,795,648

3. AppDetails Parameters



Include App Detail

- Default: `True`
- When true, generates a table entry at each interval with details of apps from the `mon <appName>;` command: `mon.striim_mon_appdetail`
- Does not include `CREATED` or `DEPLOYED` app in the output. **Inclusion of `CREATED` or `DEPLOYED` apps requires optional parameter.**

Include Describe Detail

- Default: `True`
- Enabling this provides additional information to these columns in `mon.striim_mon_appdetail`: `recoverySetting`, `checkpointDetail`, `isEncryptionEnabled`, `appCreateDate`

Include Status Detail

- Default: `True`
- Enabling this provides additional information to these columns in `mon.striim_mon_appdetail`: `deploymentOn` and `deploymentIn`

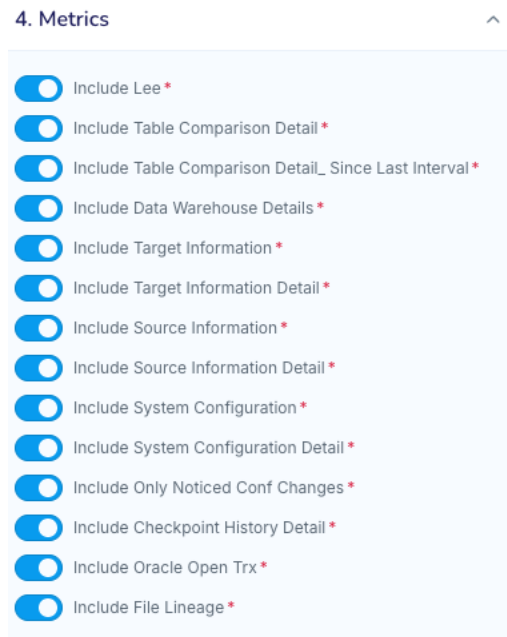
Include Created Application Detail (Not Required/Uncommon)

- Default: `False`
- When true, generates a table entry at each interval with details of apps in the `CREATED` state from the `mon <appName>;` command: `mon.striim_mon_appdetail`. This is not recommended generally.

Include Deployed Application Detail (Not Required/Uncommon)

- Default: `False`
- When true, generates a table entry at each interval with details of apps in the `CREATED` state from the `mon <appName>;` command: `mon.striim_mon_appdetail`. This is not recommended generally.

4. Metrics Parameters



Include Lee

- Default: True
- When true, generates a table entry at each interval with details of `report lee;` and `report lee+;` information: `mon.striim_mon_lee`

Include Table Comparison Detail

- Default: True
- When true, generates a table entry at each interval with details of Table Information read by Source and Target, providing a OperationName breakdown by table (one entry per table, counts of INSERT/UPDATE/DELETE/PKUPDATE/DDL) from the `mon <source/Target>;` commands: `mon.striim_mon_table_comparison`

Include Table Comparison Detail_ Since Last Interval

- Default: True
- When true, generates a table entry at each interval with details of Table Information read by Source and Target, providing a OperationName breakdown by table (one entry per table, counts of INSERT/UPDATE/DELETE/PKUPDATE/DDL) from the `mon <source/Target>;` command, but only reporting the difference that StriimWatcher noticed **since last interval**. This means that if StriimWatcher is gathering data every 5 minutes, the `_sli` entry will only contain how many records, by table, were created by OperationName in the reported 5 minute block: `mon.striim_mon_table_comparison_sli`

Include Data Warehouse Details

- Default: `False`
- When true, generates a table entry at each interval with details of Data Warehouse Targets from the `mon <Target>;` command: `mon.striim_mon_datawarehouse_detail`

Include Target Information

- Default: `True`
- When true, collects target adapter metrics (accepted events, output rates, CPU usage, lag) from `mon <target>;` command and writes to `striim_mon_target_information` table.

Include Target Information Detail

- Default: `False`
- When true, Adds full JSON output from `mon <target>;` command as `jsonoutput` field in `striim_mon_target_information` table entries.

Include Source Information

- Default: `True`
- When true, collects source adapter metrics (input rates, read lag, source freshness, CPU usage) from `mon <source>;` command and writes to `striim_mon_source_information` table.

Include Source Information Detail

- Default: `False`
- When true, adds full JSON output from `mon <source>;` command as `jsonoutput` field in `striim_mon_source_information` table entries.

Include System Configuration

- Default: `True`
- When true, collects system configuration metrics (config file parameters, memory usage, disk space) and writes to `striim_mon_system_configuration` table.

Include System Configuration Detail

- Default: `False`
- When true, adds full JSON details from configuration queries as `jsonDetail` field in `striim_mon_system_configuration` table entries.

Include Only Noticed Conf Changes

- Default: `False`
- When true, records only system configuration values that have changed since the previous monitoring interval in `striim_mon_system_configuration` table entries, reducing data volume while tracking configuration drift.

Include Checkpoint History Detail

- Default: `False`
- When true, collects checkpoint history for applications with recovery enabled, tracking source/target positions and checkpoint metadata in `striim_mon_checkpoint_history` table.

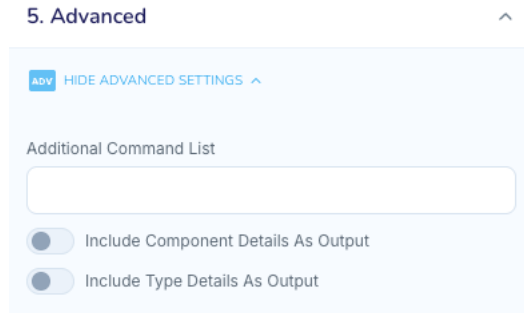
Include Oracle Open Trx

- Default: `False`
- When true, monitors open Oracle transactions from CDC sources using `show <source> OPENTRANSACTIONS;` command and writes to `striim_mon_oracle_open_trx` table.

Include File Lineage

- Default: `False`
 - When true, tracks File Lineage (e.g., Oracle GoldenGate files) via `show <source or target name> LINEAGE;` including file names, status, and timestamps in `striim_mon_file_lineage` table.
-

5. Advanced Parameters



Include Component Details As Output

Warning: May increase output of row data significantly.

Data Source: Captures raw JSON output from two types of commands:

- `mon <component>;` - Monitoring data for applications, sources, targets, and streams
- `describe <component>;` - Component configuration and property details

Collection Scope: Controlled by two configuration flags:

- `IncludeComponentDetailsAsOutput`: When true, captures `mon` and `describe` output for all sources, targets, streams, and applications
- `IncludeTypeDetailsAsOutput`: When true, captures `describe` output for TYPE definitions (schema/structure information)

If selected outputs all raw command outputs to a table called `mon.striim_mon_component_output`.

Additional Command List

A way to run and gather additional information via custom, comma-separated commands; information is stored in a component details table. Also captures output from custom commands defined in

`AdditionalCommandList` property, supporting placeholders like `%source-all%`, `%target-all%`, `%app-all%`, `%source-running%`, `%target-running%`, `%app-running%`.

Include Type Details As Output

Warning: Will increase data output and cause significant delays in execution if this is enabled.

Data Source: Collected by executing `describe <type>;` commands on all Striim TYPE definitions in the metadata repository. Types represent table schemas/structures used by CDC sources.

Change Detection Logic: Uses intelligent filtering to minimize data volume:

- **First run:** Captures all TYPE definitions and their column details

- *Subsequent runs:* Only captures types where `createdDate` has changed (indicating schema modification or type recreation)
- *Comparison:* Tracks types in `striimTypeList` to detect when `createdDate` is newer than `lastStart`

Type-to-Table Mapping: Attempts to predict which application and table each TYPE belongs to using pattern matching:

- *Pattern:* `{sourceName}_{tableName}_` (e.g., `OracleSource_EMPLOYEES_` → table `EMPLOYEES`)
- *Mapping Logic:* Executed via `mapTableNameToType()` when source table information is discovered
- *Result:* Populates `appName` and `tableName` columns with predicted values (may be empty if mapping fails)

Column Details Captured:

- `columnName:` Column name from the TYPE definition
- `columnType:` Java type (e.g., `java.lang.String`, `java.lang.Integer`, `org.joda.time.DateTime`)
- `isPK:` Boolean indicating if column is part of the primary key

Table Reference Guide

MetaData

All Striim Monitoring events will have certain metadata. The following will be on each event:

TimeStamp: The current `DateTime` of when the event is being created.

NextRun: The next `DateTime` of when `StriimWatcher` is scheduled to be run.

TotalRuns: The total number of runs `StriimWatcher` has executed.

TableName: The `StriimWatcher` table name (mon.<name>).

OperationName: Always `INSERT` for `StriimWatcher`.

ColumnCount: The number of columns in this `StriimWatcher` table.

OPERATION_TS: The current `DateTime` of when the event is being created, in long.

Events with AppName

If a Striim Monitoring table contains “App Name” as a column, then the following additional Metadata fields will be included in the event:

RelatedAppName: The related Striim App name that is being monitored.

LEE Events

For LEE monitoring events, the following additional differences are included in the Metadata:

RelatedAppName: The related Striim App name that is being monitored. For Lee events, the Source component's app is considered the RelatedAppName.

RelatedSourceAppName: The Lee Source related Striim App name that is being monitored.

RelatedTargetAppName: The Lee Target related Striim App name that is being monitored.

Table Details

striim_mon_table_runhistory (Run History Table):

Default: Always enabled (cannot be disabled).

Source Table Mapping: `mon.striim_mon_table_runhistory`

Collection Behavior

Data Source: Internal StriimWatcher application runtime tracking.

Snapshot Behavior: One row per monitoring run - tracks each execution of the StriimWatcher application.

Collection Scope: Always runs - this table cannot be disabled and is created for every monitoring cycle.

Key Metrics Captured:

- *Runtime Duration*: How long the monitoring cycle took to complete
- *Timing Information*: Start time, end time, last run, next scheduled run
- *Cluster Identification*: Cluster name and company name from configuration

Table Schema

Field Name	Description
runid	<i>Primary Key</i> - Unique bigint value for the row
batchdate	<i>Foreign Key</i> reference used by all other monitoring tables; timestamp when this batch was run (monitoring cycle start time)

runtimeEnd	The timestamp when the StriimWatcher application finished all its internal calls
runtimeDurationMS	The length of time, in milliseconds, for how long it took StriimWatcher from start to finish
clustername	The clustername from the startUp.properties file
companyname	The companyname from the startUp.properties file
lastrun	Either a default start value, or the actual last run timestamp of the StriimWatcher application
nextrun	The next planned run timestamp, based on configured value RepeatInSeconds

Key Notes

- *Always enabled* - This table cannot be disabled and is created for every monitoring run
- *Foreign key reference* - batchdate serves as the foreign key reference for all other StriimWatcher monitoring tables
- *One row per run* - Each monitoring cycle generates exactly one row
- *Performance tracking* - runtimeDurationMS helps identify slow monitoring cycles
- *Cluster identification* - clustername and companyname help identify which cluster the data came from in multi-cluster environments

striim_mon_log_watcher (Include Log Watcher)

Default: False

Source Table Mapping: mon.striim_mon_log_watcher

Collection Behavior

Data Source: Monitors log files for ERROR entries and SmartAlert WARN entries (containing "Alert Matched:"). Supports three log file types:

- striim.server.log (enabled with IncludeLogWatcher)
- striim.server.clidebug.log (enabled with IncludeDebugLogWithLogWatcher)
- striim.command.log (enabled with IncludeCommandLogWithLogWatcher)

Snapshot Behavior: Differential tracking - only processes new log entries since the last monitoring run. Position is saved separately for each log file and persists across application restarts if PreserveStriimWatcherPosition = true.

Collection Scope:

Server Log (IncludeLogWatcher):

- *ERROR entries*: All ERROR log entries except those related to system applications (e.g., System\$Notification)
- *SmartAlert entries*: WARN log entries containing "Alert Matched:" text
- *Context Capture*: For each ERROR entry, captures 20 lines before and 20 lines after the error for context analysis

Debug Log (IncludeDebugLogWithLogWatcher):

- *Parsing Logic*: Custom multi-line parsing that captures complete debug messages (including continuation lines)
- *Context Capture*: Captures the full multi-line debug message only (no additional context lines before/after)
- *File Location*: logs/striim.server.clidebug.log (only processed if file exists)

Command Log (IncludeCommandLogWithLogWatcher):

- *Command entries*: All user commands executed via API or UI
- *Format*: timestamp - username # command
- *System Command Filtering*: By default excludes "null" user (system) commands unless IncludeSystemCommandsWithLogWatcher = true
- *File Location*: logs/striim.command.log (only processed if file exists)

Processing Timeout: Log processing times out after 60 seconds per log file per monitoring run to prevent blocking.

Related Parameters

Parameter Name	Default	Description
<i>IncludeLogWatcher</i>	false	Enables monitoring of striim.server.log for ERROR and SmartAlert entries
<i>IncludeDebugLogWithLogWatcher</i>	false	Enables monitoring of striim.server.clidebug.log for ERROR entries
<i>IncludeCommandLogWithLogWatcher</i>	false	Enables monitoring of striim.command.log for user command tracking
<i>IncludeSystemCommandsWithLogWatcher</i>	false	When true, includes "null" user (system) commands from command log; when false, only user-initiated commands are captured

Key Metrics Captured

For Server Log and Debug Log ERROR Entries:

- *Error Details*: Date, server, application, log level, error message, context buffer
- *SmartAlert Details*: Alert type, alert name, related application, source/target component, metric values

For Command Log Entries:

- *Command Details*: Date, username, command text, full entry with context
- *Server Context*: Uses server name captured from server/debug log for proper server attribution
- *User Tracking*: Tracks which user executed which commands

Table Schema

Field Name	Description
errorid	<i>Primary Key</i> - Unique bigint value for the row
batchdate	<i>Foreign Key</i> - References runtime in striim_mon_table_runhistory table; timestamp when this batch was run
log_date	The parsed log entry date from the log file
server	The deployed server name (if applicable)
appName	<i>Server/Debug Log</i> : Application name related to this error (if detected); for SmartAlerts, attempts to find the related application from the source/target component listed in contextbuffertext <i>Command Log</i> : Username who executed the command
log_level	<i>Server/Debug Log</i> : For ERROR entries: ERROR; for SmartAlerts: alert name and source/target name (e.g., Source_Idle:admin.OracleCDC) <i>Command Log</i> : "striim.command.log"
message	<i>Server/Debug Log</i> : For ERROR entries: the reported error message; for SmartAlerts: the alert message (e.g., Source admin.OracleCDC: No new event received in last 72.622 (>60) seconds.) <i>Command Log</i> : The command text (portion after # in the log entry)
contextbuffertext	<i>Server/Debug Log</i> : For ERROR entries: 20 lines before the error, the error line, and 20 lines after the error for context; for SmartAlerts: the alert type followed by the full message output <i>Command Log</i> : Full log entry including timestamp, username, and command

SmartAlert Metadata

For SmartAlert monitoring events (from server/debug log only), the following additional metadata is included in the event:

For SmartAlert monitoring events, the following additional metadata is included in the event:

Metadata Key	Description
AlertType	Set to SmartAlert for SmartAlert entries; set to ERROR for error entries
SmartAlertType	The alert type detected (e.g., Application_Halted, Source_Idle)
SmartAlertName	The full alert text (e.g., Application_Halted, Medium: WEB, Message: Application admin.OracleInitialLoadApp: Application halted - Message:)

SmartAlertObject	The alert object detected (e.g., admin.OracleInitialLoadApp, admin.OracleCDC)
SmartAlertMetricValue	Metric value from the alert (if applicable)
SmartAlertMetricUnit	Metric unit from the alert (if applicable)

Key Notes

Position Tracking:

- Saves log file position separately for each log file to avoid reprocessing entries
- Persists across restarts if PreserveStriimWatcherPosition = true
- Position file: StriimWatcher_log.hist
- Tracks three separate positions: server log, debug log, command log

Server Log Processing (IncludeLogWatcher):

- System application filtering - Errors related to system applications (e.g., System\$Notification) are not reported
- Context capture - ERROR entries include 20 lines before and after for troubleshooting context
- SmartAlert detection - Automatically detects and parses SmartAlert WARN entries from the log
- Application detection - Attempts to identify the related application for both errors and SmartAlerts

Debug Log Processing (IncludeDebugLogWithLogWatcher):

- Same format as server log (timestamp @server @app -LEVEL thread class message)
- Useful for debugging when debug logging is enabled
- Shares server name context with command log

Command Log Processing (IncludeCommandLogWithLogWatcher):

- Captures all user commands executed via CLI or UI
- Multi-line command support - Handles CREATE statements and other multi-line commands
- System command filtering - By default excludes "null" user commands (set IncludeSystemCommandsWithLogWatcher = true to include)
- Server attribution - Uses server name captured from server/debug log for proper server context
- Username tracking - Tracks which user executed which commands
- Audit trail - Provides complete command history for compliance and troubleshooting

Performance:

- Timeout protection - Each log file processing times out after 60 seconds to prevent blocking the monitoring cycle
- Differential processing - Only new log entries since last run are processed
- Independent processing - Each log file is processed independently

Use Cases:

- *Server/Debug Log*: Error monitoring, SmartAlert tracking, application health monitoring, troubleshooting, log analysis
- *Command Log*: Audit trail, compliance tracking, user activity monitoring, command history, troubleshooting configuration changes

striim_mon_node_applications (Include Node Monitor)

Default: False

Source Table Mapping: `mon.striim_mon_node_applications`

Collection Behavior

Data Source: Collected via CLI command `mon;` which provides a full list of all applications in the Striim cluster.

Snapshot Behavior: Fresh output each monitoring run - captures current application list and metrics at the time of monitoring.

Collection Scope: Runs for all applications in the Striim cluster when `IncludeNodeMonitor = true`.

Table Schema

Field Name	Description
<code>monappid</code>	<i>Primary Key</i> - Unique bigint value for the row
<code>batchdate</code>	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
<code>command</code>	The equivalent console command to gather the necessary data provided here
<code>montype</code>	What type of monitoring command was run
<code>appname</code>	The Striim namespace.appName of the application on the node
<code>status</code>	The application state (e.g., <code>CREATED</code> , <code>DEPLOYED</code> , <code>RUNNING</code>)
<code>rate</code>	The rate provided from the <code>mon</code> command for each application
<code>sourcerate</code>	The source rate provided from the <code>mon</code> command for each application
<code>cpurate</code>	The CPU rate provided from the <code>mon</code> command for each application
<code>numservers</code>	Number of servers this application is running on; normally should indicate 1 or 0
<code>latestactivity</code>	The latest activity timestamp seen by the application overall

Key Notes

- *Cluster-wide view* - Provides a complete list of all applications across the cluster
- *One row per application per interval* - Each application generates one row per monitoring run
- *All application states* - Includes applications in **all** states (CREATED, DEPLOYED, RUNNING)
- *Lightweight monitoring* - Provides high-level metrics without detailed component breakdown
- *Complements app detail* - Use with `striim_mon_appdetail` for more detailed application monitoring
- *Use cases:* Application inventory, cluster-wide status overview, quick health check, application discovery

striim_mon_node_cluster (Include Node Cluster)

Default: False

Source Table Mapping: `mon.striim_mon_node_cluster`

Collection Behavior

Data Source: Collected via monitoring command that gathers Striim cluster node metrics.

Snapshot Behavior: Fresh output each monitoring run - captures current cluster node metrics at the time of monitoring.

Collection Scope: Runs for all nodes in the Striim cluster when `IncludeNodeCluster = true`.

Table Schema

Field Name	Description
<code>monnodeclusterid</code>	<i>Primary Key</i> - Unique bigint value for the row
<code>batchdate</code>	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
<code>command</code>	The equivalent console command to gather the necessary data provided here
<code>montype</code>	What type of monitoring command was run
<code>nodename</code>	The name of the node
<code>striimversion</code>	The Striim version number of the node
<code>freemem</code>	The amount of free memory on the server
<code>cpurate</code>	The current CPU rate; <i>NOTE:</i> usually 100%/core based, meaning an 8-core system can utilize 800% CPU rate

uptime	Indicates how long the Striim server has been up for
--------	--

Key Notes

- *Multi-node support* - Generates one row per cluster node per monitoring run
- *CPU rate calculation* - `cpurate` is per-core based; divide by 100 to get number of cores utilized (e.g., 800% = 8 cores fully utilized)
- *Uptime tracking* - Monitor `uptime` to detect node restarts or failures

striim_mon_node_elasticsearch (Include Node ES)

Default: False

Source Table Mapping: `mon.striim_mon_node_elasticsearch`

Collection Behavior

Data Source: Collected via monitoring command that gathers Elasticsearch cluster metrics.

Snapshot Behavior: Fresh output each monitoring run - captures current Elasticsearch metrics at the time of monitoring.

Collection Scope: Runs when Elasticsearch is configured and `IncludeNodeES = true`.

Table Schema

Field Name	Description
<code>monnodeesid</code>	<i>Primary Key</i> - Unique bigint value for the row
<code>batchdate</code>	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
<code>command</code>	The equivalent console command to gather the necessary data provided here
<code>montype</code>	What type of monitoring command was run
<code>elasticsearchReceiveThroughput</code>	Current Elasticsearch receive throughput
<code>elasticsearchTransmitThroughput</code>	Current Elasticsearch transmit throughput
<code>elasticsearchClusterStorageFree</code>	Elasticsearch storage free availability

elasticsearchClusterStorageTotal	Elasticsearch storage total
----------------------------------	-----------------------------

striim_mon_appdetail (Include App Detail)

Default: False

Related Parameters:

- *Include App Describe Detail* (Default: False) - Populates additional columns with data from `describe <app>;` command
- *Include App Status Detail* (Default: False) - Populates additional columns with data from `status <app>;` command
- *Include Created Application Detail* (Default: False) - Includes applications in `CREATED` state
- *Include Deployed Application Detail* (Default: False) - Includes applications in `DEPLOYED` state

Source Table Mapping: `mon.striim_mon_appdetail`

Collection Behavior

Data Source: Collected via command `mon <app>;` executed against each application. Additional data collected from `describe <app>;` and `status <app>;` if detail flags are enabled.

Snapshot Behavior: Fresh output each monitoring run - captures current application metrics at the time of monitoring.

Collection Scope:

- *Default:* Only includes applications in `RUNNING` state
- *With* `IncludeCreatedApplicationDetail = true:` Also includes applications in `CREATED` state
- *With* `IncludeDeployedApplicationDetail = true:` Also includes applications in `DEPLOYED` state

Key Metrics Captured:

- *Application Status:* Current state, backpressure status, recovery settings
- *Checkpoint Information:* Status and detailed position data (if describe detail enabled)
- *Deployment Information:* Server and deployment group (if status detail enabled)
- *Configuration:* Recovery, encryption settings (if describe detail enabled)

Table Schema

Field Name	Description
<code>monid</code>	<i>Primary Key</i> - Unique bigint value for the row

batchdate	<i>Foreign Key</i> - References runtime in striim_mon_table_runhistory table; timestamp when this batch was run
command	The equivalent console command to gather the necessary data provided here
appName	Application name
appStatus	Application status (e.g., RUNNING, CREATED, DEPLOYED); will not include CREATED or DEPLOYED apps unless IncludeCreatedApplicationDetail or IncludeDeployedApplicationDetail are enabled
totalInput	Total input event count from the application (what is displayed in the UI)
totalOutput	Total output event count from the application (what is displayed in the UI)
isBackpressured	Boolean indicating if the application is backpressured
isRecoveryEnabled	Boolean indicating if recovery is enabled
recoverySetting	<i>Requires</i> IncludeAppDescribeDetail = true - If recovery is enabled, the recovery setting (e.g., 1 MINUTE INTERVAL); otherwise NULL
checkpointStatus	Indicates if the checkpoint is progressing, lagging, etc.
checkpointDetail	<i>Requires</i> IncludeAppDescribeDetail = true - Displays checkpoint detailed information as a nested JSONArray (converted to String); example: [{"Source Current Position":{"CheckpointText":{"OpenSCN[5126439]-CommitSCN[5126440]-SeqNum[2]}}},"Source Name":"ADMIN:SOURCE:ORACLECDCREAD:4","Source Restart Position":{"CheckpointText":{"OpenSCN[5126439]-CommitSCN[5126440]-SeqNum[2]}}},"Application Name":"admin.PGCDCTest"}]; otherwise NULL
isEncryptionEnabled	<i>Requires</i> IncludeAppDescribeDetail = true - Boolean indicating if encryption is enabled; otherwise NULL
deploymentOn	<i>Requires</i> IncludeAppStatusDetail = true - Displays the server(s) the app is deployed on (e.g., S192_168_1_30); otherwise NULL
deploymentIn	<i>Requires</i> IncludeAppStatusDetail = true - Displays the deployment group the app is deployed in (e.g., default); otherwise NULL
appCreatedDate	<i>Requires</i> IncludeAppDescribeDetail = true - The datetime the app was created within Striim; otherwise NULL
latestActivity	The datetime of the latest activity the app has seen

App Detail: Configuration Flag Combinations

Include App Describe Detail

When enabled, populates these columns (otherwise NULL):

- `recoverySetting`
- `checkpointDetail`
- `isEncryptionEnabled`
- `appCreatedDate`

Command executed: `describe <app>;`

Include App Status Detail

When enabled, populates these columns (otherwise `NULL`):

- `deploymentOn`
- `deploymentIn`

Command executed: `status <app>;`

Include Created Application Detail

When enabled, includes applications in `CREATED` state in the monitoring results. ***Not recommended.***

Include Deployed Application Detail

When enabled, includes applications in `DEPLOYED` state in the monitoring results. ***Not recommended.***

Key Notes

- *One row per application per interval* - Each application generates one row per monitoring run
- *State filtering* - By default, only `RUNNING` applications are included; enable detail flags to include `CREATED` or `DEPLOYED` states
- *Conditional columns* - Several columns require specific detail flags to be populated; otherwise they will be `NULL`
- *Backpressure detection* - `isBackpressured` indicates if the application is experiencing backpressure (performance bottleneck)
- *Checkpoint monitoring* - `checkpointStatus` and `checkpointDetail` provide recovery position tracking
- *JSON data* - `checkpointDetail` contains JSON data stored as string; use JSON parsing functions to extract specific fields
- *Use cases:* Application health monitoring, throughput tracking, backpressure detection, checkpoint progress monitoring, deployment tracking

striim_mon_lee (Include Lee)

Default: False

Source Table Mapping: `mon.striim_mon_lee`

Collection Behavior

Data Source: Collected via CLI commands `report lee;` and `report lee+;` which measure end-to-end latency in Striim applications.

Key Metrics Captured:

- *Current Lag:* Real-time end-to-end lag measurement
- *Latency Statistics:* Minimum, maximum, and average latency over sample size
- *Source/Target Mapping:* Component names and types with attempted app name detection
- *Timing Information:* When lag was measured and source timestamp

App Name Detection: The `report lee;` command does not provide app names. StriimWatcher attempts to detect the app name by matching `sourceName` and `targetName` against known components in the metadata repository.

Table Schema

Field Name	Description
<code>monleeid</code>	<i>Primary Key</i> - Unique bigint value for the row
<code>batchdate</code>	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
<code>sourceApp</code>	Application name related to the source (if detectable); app name is not provided in the Lee report, but StriimWatcher attempts to find it by matching <code>sourceName</code>
<code>sourceName</code>	Source component name as reported by the Lee report
<code>sourceType</code>	Source component type as reported by the Lee report
<code>targetApp</code>	Application name related to the target (if detectable); app name is not provided in the Lee report, but StriimWatcher attempts to find it by matching <code>targetName</code>
<code>targetName</code>	Target component name as reported by the Lee report
<code>targetType</code>	Target component type as reported by the Lee report
<code>lagendtoend</code>	Currently measured end-to-end lag returned from <code>report lee;</code>
<code>measuredAt</code>	Timestamp when the lag was measured

sourcetime	Source timestamp returned by <code>report lee</code> ;
minlee	Minimum latency reported by Lee; represents the fastest end-to-end record creation and delivery from the sample size
maxlee	Maximum latency reported by Lee; represents the slowest end-to-end record creation and delivery from the sample size
avglee	Average latency reported by Lee; based on the sample size, represents the average time for end-to-end record creation and delivery
samplesize	Sample size used to calculate the Lee statistics

Key Notes

- *End-to-end latency tracking* - Measures time from source record creation to target delivery
- *One row per source/target pair per interval* - Each source/target combination generates one row per monitoring run; apps with multiple targets will have a row per source/target component combination.
- *App name detection* - `sourceApp` and `targetApp` may be empty if StrimWatcher cannot match component names to applications
- *Statistical analysis* - `minlee`, `maxlee`, and `avglee` provide latency distribution over the sample size
- *Use cases*: End-to-end latency monitoring, SLA compliance tracking, performance analysis, latency trend detection

striim_mon_table_comparison (Include Table Comparison Detail)

Default: False

Source Table Mapping: `mon.striim_mon_table_comparison`

Collection Behavior

Purpose: Provides cumulative comparison of operation counts between source and target tables to track replication progress and identify discrepancies.

Data Source: Aggregates operation counts from source and target component monitoring data for each table pair.

Snapshot Behavior: Cumulative totals - shows total operations since application start, not just current interval.

Collection Scope: Tracks all source/target table pairs in monitored applications. Handles complex topologies including:

- Multiple targets using routers

- CQ (Continuous Query) logic splitting workloads
- One-to-many source/target relationships

Operation Types Tracked:

- *Inserts*: New records added
- *Updates*: Existing records modified
- *Deletes*: Records removed
- *PK Updates*: Primary key changes
- *DDLs*: Schema changes (ALTER, CREATE, DROP, etc.)

Difference Calculation: $\text{diff} = \text{target} - \text{source}$ for each operation type. Note that this may be negative in instances where there was more data written to the target than was ready by the source (such as a target processing a backlog).

Table Schema

Field Name	Description
tblcompareid	<i>Primary Key</i> - Unique bigint value for the row
batchdate	<i>Foreign Key</i> - References runtime in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
appname	Application name containing the source/target pair
sourcename	Source table name
targetname	Target table name
srcNumOfDeletes	Total delete operations seen by the source for this table
srcNumOfDDLs	Total DDL operations seen by the source for this table
srcNumOfPkUpdates	Total primary key update operations seen by the source for this table
srcNumOfUpdates	Total update operations seen by the source for this table
srcNumOfInserts	Total insert operations seen by the source for this table
tgtNumOfDeletes	Total delete operations processed by the target for this table
tgtNumOfDDLs	Total DDL operations processed by the target for this table
tgtNumOfPkUpdates	Total primary key update operations processed by the target for this table
tgtNumOfUpdates	Total update operations processed by the target for this table
tgtNumOfInserts	Total insert operations processed by the target for this table
diffNumOfDeletes	Difference in delete operations (<code>tgtNumOfDeletes - srcNumOfDeletes</code>)

diffNumOfDDLs	Difference in DDL operations (tgtNumOfDDLs - srcNumOfDDLs)
diffNumOfPkUpdates	Difference in primary key update operations (tgtNumOfPkUpdates - srcNumOfPkUpdates)
diffNumOfUpdates	Difference in update operations (tgtNumOfUpdates - srcNumOfUpdates)
diffNumOfInserts	Difference in insert operations (tgtNumOfInserts - srcNumOfInserts)
sourceComponentName	Source component name collecting the data
targetComponentName	Target component name writing the data

Key Notes

- *Cumulative tracking* - Shows total operations since application start, not interval-based
- *One row per table pair per interval* - Each source/target table combination generates one row per monitoring run
- *Initial Load vs CDC* - Useful for both initial load (count-only) and CDC (operation-type breakdown)
- *Use cases*: Replication progress tracking, lag detection, data consistency validation, initial load monitoring

striim_mon_table_comparison_sli (Include Table Comparison Detail _ Since Last Interval)

Default: False

Source Table Mapping: `mon.striim_mon_table_comparison_sli`

Collection Behavior

Purpose: Provides interval-based comparison of operation counts between source and target tables to track replication throughput and detect issues.

Data Source: Calculates the difference in operation counts between the current run and the previous run for each table pair.

Differential Tracking:

- *First run*: No data collected (nothing to compare against)
- *Subsequent runs*: Compares current operation counts against previous run to calculate interval deltas

Operation Types Tracked and Difference Calculation: these are similar to `mon.striim_mon_table_comparison` except it will contain interval-based counts.

Table Schema

Field Name	Description
tblcomparesliid	<i>Primary Key</i> - Unique bigint value for the row
batchdate	<i>Foreign Key</i> - References runtime in striim_mon_table_runhistory table; timestamp when this batch was run
timeSinceLastBatch	Number of seconds elapsed since the last monitoring run
appname	Application name containing the source/target pair
sourcename	Source table name
targetname	Target table name
srcNumOfDeletes_sli	Delete operations seen by the source for this table since last interval; no data on first run
srcNumOfDDLs_sli	DDL operations seen by the source for this table since last interval; no data on first run
srcNumOfPkUpdates_sli	Primary key update operations seen by the source for this table since last interval; no data on first run
srcNumOfUpdates_sli	Update operations seen by the source for this table since last interval; no data on first run
srcNumOfInserts_sli	Insert operations seen by the source for this table since last interval; no data on first run
tgtNumOfDeletes_sli	Delete operations processed by the target for this table since last interval; no data on first run
tgtNumOfDDLs_sli	DDL operations processed by the target for this table since last interval; no data on first run
tgtNumOfPkUpdates_sli	Primary key update operations processed by the target for this table since last interval; no data on first run
tgtNumOfUpdates_sli	Update operations processed by the target for this table since last interval; no data on first run
tgtNumOfInserts_sli	Insert operations processed by the target for this table since last interval; no data on first run
diffNumOfDeletes_sli	Difference in delete operations since last interval (tgtNumOfDeletes_sli - srcNumOfDeletes_sli); no data on first run
diffNumOfDDLs_sli	Difference in DDL operations since last interval (tgtNumOfDDLs_sli - srcNumOfDDLs_sli); no data on first run

diffNumOfPkUpdates_sli	Difference in primary key update operations since last interval (tgtNumOfPkUpdates_sli - srcNumOfPkUpdates_sli); no data on first run
diffNumOfUpdates_sli	Difference in update operations since last interval (tgtNumOfUpdates_sli - srcNumOfUpdates_sli); no data on first run
diffNumOfInserts_sli	Difference in insert operations since last interval (tgtNumOfInserts_sli - srcNumOfInserts_sli); no data on first run
sourceComponentName	Source component name collecting the data
targetComponentName	Target component name writing the data

Key Notes

- *Interval-based tracking* - Shows operations processed in the current interval, not cumulative totals
- *No first run data* - Requires at least two monitoring runs to generate data (nothing to compare on first run)
- *One row per table pair per interval* - Each source/target table combination generates one row per monitoring run (after first run)
- *Throughput analysis* - Use `timeSinceLastBatch` to calculate operations per second
- *Complements cumulative table* - Use with `striim_mon_table_comparison` for both cumulative and interval views

striim_mon_datawarehouse_detail (Include Data Warehouse Details)

Default: False

Source Table Mapping: `mon.striim_mon_datawarehouse_detail`

Collection Behavior

Data Source: Collected via command `mon <target>`; executed against target components that are data warehouse adapters (e.g., BigQuery, Snowflake, Databricks).

Collection Scope: Only runs for target components that are data warehouse adapters and include data warehouse-specific metrics in their `mon <target>`; output.

Key Metrics Captured:

- *Configuration Details:* Project ID, mode (APPENDONLY/MERGE), streaming upload settings, batch settings
- *Batch Statistics:* Total batches created/uploaded/queued/ignored, partition pruning

- *Performance Metrics:* Average and min/max integration times, compaction times, merge times, upload times
- *Last Batch Details:* Event counts, inserts/updates/deletes, batch size, DDL operations
- *Integration Task Timing:* Detailed breakdown of last batch processing times

Table Schema

Field Name	Description
dwdid	<i>Primary Key</i> - Unique bigint value for the row
batchdate	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
appName	Application name containing the data warehouse target
sourceName	Associated source component name
targetName	Associated target component name
target_adaptername	Target adapter name (e.g., <code>BigQueryWriter</code> , <code>SnowflakeWriter</code> , <code>DatabricksWriter</code>)
projectId	Project ID (if applicable, e.g., GCP project for BigQuery)
Mode	Data warehouse mode: <code>APPENDONLY</code> or <code>MERGE</code>
streamingUpload	Boolean indicating if streaming upload is enabled
StreamingConfiguration	Streaming configuration details, if enabled
optimizedMerge	Boolean indicating if optimized merge is enabled
batch_event_count	Target adapter's event count batch setting
batch_interval	Target adapter's interval batch setting
total_batches_created	Total number of batches created
partition_pruned_batches	Number of batches where partition pruning was applied
last_successful_merge_time	Timestamp of the last successful merge operation
total_batches_ignored	Total number of batches ignored (e.g., empty batches)
max_integration_time_ms	Maximum integration time in milliseconds
avg_in_mem_compaction_time_ms	Average in-memory compaction time in milliseconds
avg_batch_size_bytes	Average batch size in bytes
avg_event_count_per_batch	Average number of events per batch
min_integration_time_ms	Minimum integration time in milliseconds

total_batches_queued	Total number of batches currently queued
avg_compaction_time_ms	Average compaction time in milliseconds
avg_waiting_time_in_queue_ms	Average waiting time in queue in milliseconds
avg_integration_time_ms	Average total integration time in milliseconds
total_batches_uploaded	Total number of batches uploaded to data warehouse
avg_merge_time_ms	Average merge time in milliseconds
avg_stage_resources_mgmt_time_ms	Average stage resources management time in milliseconds
avg_upload_time_ms	Average upload time in milliseconds
last_batch_updates	Number of update operations in the last batch
last_batch_event_count	Number of events in the last batch
last_batch_inserts	Number of insert operations in the last batch
last_batch_max_record_size_bytes	Maximum record size in the last batch in bytes
last_batch_total_events_merged	Total events merged in the last batch
last_batch_ddls	Number of DDL operations in the last batch
last_batch_sequence_number	Sequence number of the last batch
last_batch_size_bytes	Size of the last batch in bytes
last_batch_deletes	Number of delete operations in the last batch
last_batch_pk_updates	Number of primary key update operations in the last batch
last_batch_accumulation_time_ms	Time spent accumulating events for the last batch in milliseconds
last_batch_compaction_time_ms	Compaction time for the last batch in milliseconds
last_batch_stage_resources_mgmt_time_ms	Stage resources management time for the last batch in milliseconds
last_batch_upload_time_ms	Upload time for the last batch in milliseconds
last_batch_merge_time_ms	Merge time for the last batch in milliseconds
last_batch_in_mem_compaction_time_ms	In-memory compaction time for the last batch in milliseconds
last_batch_pk_update_time_ms	Primary key update time for the last batch in milliseconds
last_batch_ddl_execution_time_ms	DDL execution time for the last batch in milliseconds

last_batch_total_integration_time_ms	Total integration time for the last batch in milliseconds
targetComponentName	Data warehouse target component name

Key Notes

- *Data warehouse targets only* - Only applicable to BigQuery, Snowflake, Databricks, and similar data warehouse adapters
- *Performance analysis* - Rich metrics for analyzing batch processing performance and identifying bottlenecks
- *Queue monitoring* - increasing or large `total_batches_queued` and `avg_waiting_time_in_queue_ms` indicate backlog issues
- *Batch size tracking* - Monitor `avg_batch_size_bytes` and `last_batch_size_bytes` for optimization
- *Operation breakdown* - Last batch details show inserts/updates/deletes for data change tracking
- *Timing breakdown* - Integration task timing fields help identify which stage is slowest (compaction, upload, merge, etc.)
- *Use cases*: Data warehouse performance tuning, batch optimization, queue monitoring, throughput analysis, bottleneck identification

striim_mon_target_information (Include Target Information / Include Target Information Detail)

Default: True

Related Properties:

- *Include Target Information Detail* (Default: False) - Populates `jsonoutput` column with full JSON from `mon <target>; command`

Source Table Mapping: `mon.striim_mon_target_information`

Collection Behavior

Data Source: Collected via command `mon <target>;` executed against each target component in monitored applications.

Data Parsing: Many fields are parsed from formatted strings:

- Comma-separated numbers: `'4,180'` → 4180
- Percentages: `'0.879%'` → 0.879
- Units preserved: `'56.018 sec'`, `'0MB/s'` kept as strings

Table Schema

Field Name	Description
montgtinford	<i>Primary Key</i> - Unique bigint value for the row
batchdate	<i>Foreign Key</i> - References runtime in striim_mon_table_runhistory table; timestamp when this batch was run
appName	Application name containing the target component
componentName	Target component name being monitored
accepted	Total events accepted by the target; parsed from comma-separated string (e.g., '4,180' → 4180)
noOfEventsAcceptedPerInterval	Events accepted in the current monitoring interval
acceptedRate	Rate of event acceptance (events per second)
input_count	Total input events to the target; parsed from comma-separated string (e.g., '4,239' → 4239)
inputRate	Input event rate (events per second)
output_count	Total output events from the target; parsed from comma-separated string (e.g., '4,239' → 4239)
event_rate	General processing rate (events per second)
targetAcked	Events acknowledged by the target; parsed from comma-separated string (e.g., '4,180' → 4180)
targetOutput	Events output to the target; parsed from comma-separated string (e.g., '4,180' → 4180)
targetRate	Target output rate (events per second)
cpu	CPU usage value (e.g., 0.00879)
cpuRatePerNode	CPU rate per node; parsed from percentage string (e.g., '0.11%' → 0.11)
cpuRate	Overall CPU rate percentage; parsed from percentage string (e.g., '0.879%' → 0.879)
discardedEventCount	Count of discarded events - indicates potential data loss
numberOfEventsSeenPerMonitorSnapshotInterval	Events seen in the monitoring snapshot interval

lastEventWriteAge	Age of the last written event - indicates staleness (e.g., '56.018 sec'); kept as STRING due to units
latestActivity	Timestamp of the latest activity on the target
maxLeeFromAllSources	Maximum lag from all sources - indicates lag/freshness issues (e.g., '5,384,403.239 sec'); kept as STRING due to units and complex format
numServers	Number of servers
montimestamp	Monitoring timestamp when metrics were collected
writeBytes	Bytes written per second (e.g., '0MB/s'); kept as STRING due to units
jsonoutput	Full JSON output from mon <target>; command; only populated if IncludeTargetInformationDetail = true, otherwise NULL

Configuration Flag Combinations

IncludeTargetInformation	IncludeTargetInformationDetail	Result
false	N/A	No target information collected
true	false	Target metrics collected; jsonoutput = NULL
true	true	Target metrics collected; jsonoutput populated with full JSON

Key Notes

- *Default enabled* - Target information collection is enabled by default (IncludeTargetInformation = true)
- *Discarded Event Detection* - Monitor discardedEventCount for non-zero values indicating potential target table mapping issues.
- *Lag monitoring* - maxLeeFromAllSources and lastEventWriteAge indicate target lag and staleness
- *String vs numeric fields* - Some fields preserved as strings to retain units (sec, MB/s, etc.)
- *JSON detail* - Enable IncludeTargetInformationDetail = true to capture full mon <target>; output for advanced diagnostics

striim_mon_source_information (Include Source Information / Include Source Information Detail)

Default: True

Related Properties:

- *Include Source Information Detail* (Default: False) - Populates `jsonoutput` column with full JSON from `mon <source>; command`

Source Table Mapping: `mon.striim_mon_source_information`

Collection Behavior

Data Source: Collected via command `mon <source>;` executed against each source component in monitored applications.

Data Parsing: Many fields are parsed from formatted strings:

- Comma-separated numbers: `'57,253'` → 57253
 - Percentages: `'0.235%'` → 0.235
 - Time durations: `'08H:02M:32S'` → 482 minutes
-

Table Schema

Field Name	Description
<code>monsrcinfoid</code>	<i>Primary Key</i> - Unique bigint value for the row
<code>batchdate</code>	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
<code>appName</code>	Application name containing the source component
<code>componentName</code>	Source component name being monitored
<code>input_count</code>	Total input events to the source; parsed from comma-separated string (e.g., <code>'57,253'</code> → 57253)
<code>inputRate</code>	Input event rate (events per second)
<code>sourceInput</code>	Source input events; parsed from comma-separated string (e.g., <code>'57,253'</code> → 57253)

sourceRate	Source event rate (events per second)
event_rate	General processing rate
numberOfEventsSeenPerMonitorSnapshotInterval	Events seen in the monitoring snapshot interval
cpu	CPU usage value (e.g., 0.00235)
cpuRatePerNode	CPU rate per node; parsed from percentage string (e.g., '0.029%' → 0.029)
cpuRate	Overall CPU rate percentage; parsed from percentage string (e.g., '0.235%' → 0.235)
lastEventReadAge	Age of the last read event - indicates staleness (e.g., '0.24 sec')
latestActivity	Timestamp of the latest activity on the source
readLag	Read lag value - indicates how far behind the source is from the database; parsed from comma-separated string (e.g., '156,800' → 156800)
readTimestamp	Read timestamp - the timestamp of the last read event from the source database
sourceFreshness	Source freshness indicator - how fresh the data is from the source (e.g., '08H:02M:32S' or '1D:08H:02M:32S'); original string format preserved
sourceFreshnessMinutes	Source freshness converted to total minutes; calculated from sourceFreshness field; format supports days, hours, minutes, seconds (e.g., '08H:02M:32S' → 482 minutes, '1D:08H:02M:32S' → 1922 minutes)
numServers	Number of servers
montimestamp	Monitoring timestamp when metrics were collected
jsonoutput	Full JSON output from mon <source>; command; only populated if IncludeSourceInformationDetail = true, otherwise NULL

Key Metrics Explained

Lag and Freshness Metrics

Metric	Purpose	Example
--------	---------	---------

sourceFreshness	Time duration behind the source database	'08H:02M:32S' (8 hours, 2 minutes, 32 seconds behind)
sourceFreshnessMinutes	Same as sourceFreshness but in total minutes for easier querying	482 (8 hours 2 minutes = 482 minutes)
lastEventReadAge	How long ago the last event was read	'0.24 sec' (very recent)

Configuration Flag Combinations

IncludeSourceInformation	IncludeSourceInformationDetail	Result
false	N/A	No source information collected
true	false	Source metrics collected; jsonoutput = NULL
true	true	Source metrics collected; jsonoutput populated with full JSON

Key Notes

- *Default enabled* - Source information collection is enabled by default (`IncludeSourceInformation = true`)
- *String parsing* - Many numeric fields are parsed from formatted strings (commas, percentages, time durations)
- *JSON detail* - Enable `IncludeSourceInformationDetail = true` to capture full `mon <source>`; output for advanced diagnostics

striim_mon_system_configuration (Include System Configuration / Include System Configuration Detail / Include Only Noticed Conf Changes)

Default: True

Related Properties:

- *Include System Configuration Detail* (Default: False) - Populates `jsonDetail` column with full JSON output
- *Include Only Noticed Conf Changes* (Default: False) - Filters entries to only include changed values

Source Table Mapping: `mon.striim_mon_system_configuration`

Collection Behavior

Data Source: Collected from three system monitoring categories:

1. CONFIG_FILE - Configuration Files

Monitors configuration files and stores all key-value pairs as JSON:

- `conf/startUp.properties`
- `conf/agent.conf`
- `conf/windowsAgent/yajsw_agent/conf/*.conf` (all `.conf` files in this directory)

2. MEMORY - JVM and Physical Memory Metrics

Collects 8 memory metrics (all values in bytes):

- JVM Max Memory, JVM Total Memory, JVM Free Memory, JVM Used Memory
- Total Physical Memory, Free Physical Memory
- Total Swap Space, Free Swap Space

3. DISK_SPACE - Filesystem Space Metrics

Monitors all filesystem roots with metrics:

- `total_bytes`, `free_bytes`, `usable_bytes`, `used_bytes`

Change Detection: Uses `previousConfigValues` Map to track changes across runs:

- Compares current values against previous run
- Sets `valueChanged = true` if value differs or is first time seen
- Persistence enabled: Previous values saved to disk when `PreserveStriimWatcherPosition=true`

Filtering Behavior:

- `IncludeOnlyNoticedConfChanges = false` (default): Records all configuration values every interval
- `IncludeOnlyNoticedConfChanges = true`: Only records entries where `valueChanged = true`

Table Schema

Field Name	Description
<code>monsysconfigid</code>	<i>Primary Key</i> - Unique bigint value for the row
<code>batchdate</code>	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
<code>configType</code>	Type of configuration entry: <code>CONFIG_FILE</code> (configuration files like <code>startUp.properties</code> , <code>agent.conf</code>), <code>MEMORY</code> (JVM and physical memory metrics), or <code>DISK_SPACE</code> (filesystem space metrics)

parameterName	Parameter name, metric name, filename, or filesystem path depending on configType • <i>CONFIG_FILE</i>: filename (e.g., startUp.properties) • <i>MEMORY</i>: metric name (e.g., JVM Max Memory (bytes)) • <i>DISK_SPACE</i>: filesystem path (e.g., /, C:\)
parameterValue	Value of the parameter or metric • <i>CONFIG_FILE</i>: JSON object with key-value pairs from the config file • <i>MEMORY</i>: numeric value in bytes • <i>DISK_SPACE</i>: JSON object with total_bytes, free_bytes, usable_bytes, used_bytes
valueChanged	Indicates whether this value changed from the previous monitoring run; true if changed or first time seen, false if unchanged; used for change detection when IncludeOnlyNoticedConfChanges is enabled
jsonDetail	Full JSON output from system configuration collection methods (getConfDetails, getSystemMemory, getSystemSpace); only populated if IncludeSystemConfigurationDetail = true, otherwise NULL

Configuration Flag Combinations

IncludeSystemConfiguration	IncludeSystemConfigurationDetail	IncludeOnlyNoticedConfChanges	Result
false	N/A	N/A	No data collected
true	false	false	All config values recorded every interval; jsonDetail = NULL
true	true	false	All config values recorded every interval; jsonDetail populated

true	false	true	Only changed values recorded; jsonDetail = NULL
true	true	true	Only changed values recorded; jsonDetail populated

Key Notes

- *Default enabled* - Unlike most monitoring features, this is enabled by default (IncludeSystemConfiguration = true)
- *Data volume control* - Use IncludeOnlyNoticedConfChanges = true to reduce data volume
- *Change tracking* - valueChanged column enables configuration drift detection
- *JSON querying* - Use database-specific JSON functions to extract specific fields from parameterValue and jsonDetail columns
- *Typical row count* - Default config generates ~15-20 rows per interval (varies by number of config files and filesystems)
- *Use cases*: Configuration drift detection, capacity monitoring (disk/memory), system health tracking

striim_mon_checkpoint_history (Include System Configuration / Include Checkpoint History Detail)

Default: False

Source Table Mapping: mon.striim_mon_checkpoint_history

Collection Behavior

Data Source: Collected from Striim's internal MetadataRepository API by running SHOW <appName> CHECKPOINT HISTORY; - Only captures data for applications with *recovery enabled*.

Differential Tracking: Uses change detection to avoid duplicate events:

- *First run:* Establishes baseline by collecting all existing checkpoints but creates *no events*

- *Subsequent runs*: Compares current checkpoints against previous run and creates events only for *NEW* checkpoints
- *Purpose*: Provides historical audit trail of new checkpoints without duplicating the same checkpoint across multiple monitoring intervals

Serial Number Ordering: Lower `serialNo` values indicate *more recent* checkpoints (Striim assigns incrementing serial numbers with each new checkpoint).

Position Tracking: Captures both source and target positions at checkpoint time:

- `sourcePositionSummary`: Contains SOURCE RESTART POSITION and SOURCE CURRENT POSITION with format varying by source type (Oracle SCN, SQL Server LSN, PostgreSQL LSN, MySQL binlog position)
- `targetPositionSummary`: Shows TARGET ACKNOWLEDGED POSITION (may be `NULL` if target doesn't support position tracking)

Timestamps: `checkpointRecordedTime` reflects when *Striim created* the checkpoint, while `batchdate` reflects when *StriimWatcher discovered* it (these will differ).

Table Schema

Field Name	Description
<code>monchkpthistid</code>	<i>Primary Key</i> - Unique bigint value for the row
<code>batchdate</code>	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
<code>appName</code>	Application name with recovery enabled
<code>serialNo</code>	Serial number of the checkpoint entry; <i>lower numbers indicate more recent checkpoints</i> ; assigned by Striim and increments with each new checkpoint
<code>sourcePositionSummary</code>	Summary of source position at checkpoint time; contains SOURCE RESTART POSITION and SOURCE CURRENT POSITION including table names, commit SCNs, sequence values, or other source-specific position markers; format varies by source type (Oracle CDC, SQL Server CDC, PostgreSQL, etc.)
<code>targetPositionSummary</code>	Summary of target position at checkpoint time; contains TARGET ACKNOWLEDGED POSITION showing what data has been successfully written and acknowledged by the target; may be <code>NULL</code> if target position is not available; format varies by target type
<code>checkpointType</code>	Type of checkpoint; common values: <code>normal</code> (regular checkpoints), <code>manual</code> (user-initiated), <code>recovery</code> (created during recovery)
<code>checkpointRecordedTime</code>	Timestamp when this checkpoint was recorded by Striim; this is when the checkpoint was <i>created</i> , not when it was discovered by StriimWatcher

Key Notes

- Only applications with recovery enabled appear in this table
- Position summaries are text strings (not JSON) with format varying by source type
- Useful for recovery point monitoring, checkpoint frequency analysis, and source-target lag detection
- Persistence enabled: Previous run data is saved to disk when `PreserveStriimWatcherPosition=true` and restored on restart
- No data collected for applications without recovery enabled

striim_mon_oracle_open_trx (Include Oracle Open Trx)

Default: False

Source Table Mapping: `mon.striim_mon_oracle_open_trx`

Collection Behavior

Data Source: Collected via command `show <source_component> OPENTRANSACTIONS;` executed against each SOURCE component.

Snapshot Behavior: Fresh output each monitoring run - only currently open transactions at the time of monitoring are included. This is not cumulative or historical tracking.

Collection Scope: Command is executed against all SOURCE components, but only Oracle CDC or OJet sources will return successful results. If the command returns "Failure" status (no open transactions or not applicable for non-Oracle sources), no entries are added for that component.

Transaction Tracking: Each row represents a single open Oracle transaction with details including:

- *transactionId*: Transaction ID (e.g., "4.8.11465")
- *numOfOps*: Number of operations in the transaction
- *sequenceNum*: Sequence number from Oracle
- *startscn*: Start SCN (System Change Number)
- *rbaBlock*: RBA (Redo Byte Address) block number
- *threadNum*: Thread number
- *montimestamp*: Transaction start timestamp (stored as DateTime)

Table Schema

Field Name	Description
monopentrxid	<i>Primary Key</i> - Unique bigint value for the row

batchdate	<i>Foreign Key</i> - References runtime in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
appName	Application name containing the Oracle CDC source component
componentName	Oracle CDC source component name being monitored
transactionId	Oracle transaction ID (e.g., 4.8.11465)
numOfOps	Number of operations in the open transaction
sequenceNum	Sequence number of the transaction
startscn	Start SCN (System Change Number) of the transaction
rbaBlock	RBA (Redo Byte Address) block number
threadNum	Oracle thread number
montimestamp	Timestamp when the transaction started

Key Notes

- Only applicable to Oracle CDC/OJet sources (not SQL Server, PostgreSQL, MySQL, etc.)
- Useful for monitoring long-running transactions
- Empty result set is normal when no transactions are open
- Transaction age can be calculated by comparing `timestamp` (when transaction started) to `batchdate` (when monitoring ran)
- No differential tracking - captures current state at each monitoring interval

striim_mon_file_lineage (Include File Lineage)

Default: False

Source Table Mapping: `mon.striim_mon_file_lineage`

Collection Behavior

Data Source: Collected via command `show <component> LINEAGE;` executed against each source AND target component in the application (not limited to CDC sources).

Differential Tracking: Uses change detection to provide initial state visibility and track changes:

- *First run:* Collects all file lineage entries and *outputs ALL entries* to establish initial state in the monitoring table
- *Subsequent runs:* Compares current file lineage against previous run

- *Events Created*: Only for NEW files or status changes (e.g., file transitions from PROCESSING to COMPLETED)

Match Logic: Files are considered the same if all of the following match:

- `appName + componentName + fileName + file_status`

If any of these fields differ, the entry is considered NEW or CHANGED and an event is created.

File Status Values:

- **PROCESSING**: File is currently being read by the source/written by the target
- **COMPLETED**: File has been fully processed
- Other status values may exist depending on the source/target type

Collection Scope:

- Runs for *both SOURCE and TARGET components* (not just sources)
- Only processes components where the LINEAGE command returns "Success" status
- If the command returns "Failure" status (component doesn't support lineage tracking), no entries are added for that component

Table Schema

Field Name	Description
<code>monfilelineageid</code>	<i>Primary Key</i> - Unique bigint value for the row
<code>batchdate</code>	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
<code>appName</code>	Application name containing the source component
<code>componentName</code>	Source component name being monitored
<code>fileName</code>	Name of the trail file or CDC file being processed (e.g., <code>ea000082385</code>); unique identifier for the file in the lineage
<code>status</code>	Processing status of the file; common values: PROCESSING (currently being read), COMPLETED (fully processed)
<code>directoryName</code>	Full directory path where the file is located (e.g., <code>/ogg/ogg191/dirdat</code>); physical location of the trail file on the source system
<code>fileCreationTime</code>	Timestamp when the file was created on the source system; helps track when new trail files are generated
<code>numberOfEvents</code>	Total number of events/records contained in this file; value of 0 indicates an empty file or a file with only metadata

firstEventTimestamp	Timestamp of the first event in the file; NULL or N/A if the file contains no events; represents the earliest data change captured in this file
lastEventTimestamp	Timestamp of the last event in the file; NULL or N/A if the file contains no events; represents the most recent data change captured in this file
wrapNumber	Wrap number for the trail file sequence; used in GoldenGate to track file sequence across wraps; typically 1 for most deployments
sequenceNumber	Sequence number for the trail file; may be NULL if not applicable; used to order files within a wrap

Key Notes

- Primarily used for Oracle GoldenGate trail file tracking, but applicable to any file-based CDC source
- Tracks file processing progress through status transitions (new file appears → PROCESSING → COMPLETED)
- numberOfEvents of 0 indicates an empty file or metadata-only file
- firstEventTimestamp and lastEventTimestamp are NULL or N/A for empty files
- wrapNumber and sequenceNumber are GoldenGate-specific fields for tracking file sequence across wraps
- Useful for monitoring file processing lag, detecting stuck files, and tracking when new trail files are generated
- Persistence enabled: Previous run data is saved to disk when PreserveStriimWatcherPosition=true and restored on restart

5. Advanced Parameters

striim_mon_component_output (Include Component Details As Output)

Default: False

Source Table Mapping: mon.striim_mon_component_output

Collection Behavior

Data Source: Captures raw JSON output from Striim CLI commands for detailed component analysis and diagnostics.

When Enabled (IncludeComponentDetailsAsOutput = true):

- mon *commands:* Application, Source, Stream, and Target monitoring output
- describe *commands:* Source and Target configuration details

Additional Data Collection Triggers:

This table will also be populated when either of the following options are enabled, even if `IncludeComponentDetailsAsOutput = false`:

1. `IncludeTypeDetailsAsOutput = true`
 - Captures `describe <type>` output for TYPE definitions
 - Only on first run OR when TYPE `createdDate` changes
 - Enables schema change tracking
2. `AdditionalCommandList` *is configured*
 - Captures output from custom commands specified in the list
 - See "Additional Command List" section for details

Output Structure by Source

Standard Component Output (`IncludeComponentDetailsAsOutput = true`)

Column	Value
appName	Application name containing the component
componentName	Full component name (e.g., admin.MySource)
command	mon or describe
type	APP, SOURCE, STREAM, or TARGET
jsondata	Raw JSON response from the command

Type Details Output (`IncludeTypeDetailsAsOutput = true`)

Column	Value
appName	Empty/blank
componentName	TYPE name (e.g., OracleSource_EMPLOYEES_)
command	describe
type	TYPE
jsondata	Raw JSON schema definition

Custom Command Output (`AdditionalCommandList` configured)

Column	Value
appName	Empty/blank
componentName	<i>Full command executed</i> (e.g., mon admin.MySource memorysize)
command	Command type: mon, describe, status, report, list, meter, show, or usage
type	CUSTOM
jsondata	Raw JSON response from the command

Table Schema

Field Name	Description
moncomoutid	<i>Primary Key</i> - Unique bigint value for the row
batchdate	<i>Foreign Key</i> - References runtime in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
appName	Application name for the monitored component (may be blank for TYPE or CUSTOM entries)
componentName	Component name being monitored (for CUSTOM type, this is the full command executed)
command	Command type executed: <code>mon</code> , <code>describe</code> , <code>status</code> , <code>report</code> , <code>list</code> , <code>meter</code> , <code>show</code> , or <code>usage</code>
type	Component type: <code>APP</code> , <code>SOURCE</code> , <code>STREAM</code> , <code>TARGET</code> , <code>TYPE</code> , or <code>CUSTOM</code>
jsondata	Raw JSON output from the API response; useful for tracking component properties, configurations, and detailed diagnostics

Use Cases

1. Configuration Change Tracking

- Store `describe` output over time to detect configuration drift
- Compare `jsondata` between runs to identify property changes

2. Advanced Diagnostics

- Capture detailed component metrics not available in structured tables
- Query JSON fields using BigQuery/PostgreSQL JSON functions

3. Custom Monitoring

- Use `AdditionalCommandList` to capture specialized metrics
- Monitor memory usage, component status, or custom reports

4. Schema Evolution

- Track TYPE definition changes via `describe <type>` output
 - Audit when and how table schemas change
-

Key Notes

- *Advanced feature* - Not required for standard monitoring (default: false); not recommended in most instances.
- *Data volume impact* - Stores complete JSON responses; can significantly increase storage
- *Multiple triggers* - Table is populated by three different mechanisms (component details, type details, additional commands)
- *JSON querying* - Use database-specific JSON functions to extract specific fields from `jsondata` column
- *Change detection* - describe output for components only stored when configuration changes (via `upsertComponentDescribe()`)

Additional Command List

Source Table Mapping: `mon.striim_mon_component_output`

Command Syntax:

```
<command1>{optionalIntervalInSeconds};<command2>{optionalIntervalInSeconds};...
```

Components:

- *Command:* Any valid Striim CLI command (see allowed commands below)
- *Interval (optional):* Execution frequency in seconds, enclosed in { }
 - Must be a multiple of `RepeatInSeconds` property
 - If omitted, uses `RepeatInSeconds` as default interval
 - Example: { 600 } = execute every 600 seconds

Allowed Commands: `mon, describe, status, report, list, meter, show, usage`

Dynamic Placeholders

Replace component/app names with placeholders to execute commands across multiple targets:

Placeholder	Replacement
<code>%source-all%</code>	Every source component (all apps, any state)
<code>%source-running%</code>	Every source component in RUNNING apps only
<code>%target-all%</code>	Every target component (all apps, any state)
<code>%target-running%</code>	Every target component in RUNNING apps only
<code>%app-all%</code>	Every application (any state)

%app-running%	Every RUNNING application only
---------------	--------------------------------

Examples for Additional Commands List

Example 1: Multiple Commands with Different Intervals

```
mon admin.StriimWatcher memorysize{600};usage;
```

- `mon admin.StriimWatcher memorysize` executes every 600 seconds
- `usage` executes every RepeatInSeconds (default interval)

Example 2: Monitor All Source Memory

```
mon %source-all% memorysize{900}
```

- Executes `mon <source> memorysize` for every source component
- Runs every 900 seconds (15 minutes)

Example 3: Monitor Running App Status

```
status %app-running%{300}
```

- Executes `status <app>` for every running application
- Runs every 300 seconds (5 minutes)

Output Structure

Results are written to `mon.striim_mon_component_output` with the following column values:

Column	Value
appName	Empty/blank
componentName	The <i>full command</i> that was executed (e.g., <code>mon admin.MySource memorysize</code>)
command	The <i>command type</i> (e.g., <code>mon</code> , <code>describe</code> , <code>status</code>)
type	CUSTOM
jsondata	Raw JSON response from the command

striim_mon_table_column_detail (Include Type Details As Output)

Default: False

Source Table Mapping: `mon.striim_mon_table_column_detail`

Collection Behavior

Data Source: Collected by examining all TYPE definitions in the Striim metadata repository.

Snapshot Behavior: Differential tracking - by default, only produces output entries on first run and when a type's `createdDate` has changed.

Collection Scope:

- Examines all types in the Striim system
- *Excludes:* Types with namespace `System` or `Global`, and types created by StriimWatcher itself
- *App name detection:* If `IncludeAppDetail = true`, attempts to match types to applications using app name + table name → type matching logic

Memory Considerations: This feature may be more memory intensive as it examines all types in the system to create a comprehensive data map.

Key Metrics Captured:

- *Schema Information:* Column names, data types, primary key indicators
- *Type Metadata:* Type name, creation date
- *Relationship Mapping:* Predicted application and table names

Table Schema

Field Name	Description
montblcoldtlid	<i>Primary Key</i> - Unique bigint value for the row
batchdate	<i>Foreign Key</i> - References <code>runtime</code> in <code>striim_mon_table_runhistory</code> table; timestamp when this batch was run
typeName	The Striim type name that was produced related to the table
appName	The predicted application name that utilizes this type; attempts to match by utilizing app name + table name → type matching logic; blank if <code>IncludeAppDetail = false</code>
tableName	The predicted table name that utilizes this type; attempts to match by utilizing app name + table name → type matching logic
createdDate	When the type was created; by default, StriimWatcher only produces output entries on first run and when a type's <code>createdDate</code> has changed
columnName	The column name from the table
columnType	The column type from the table
isPK	Boolean indicating whether this column is a primary key column

Key Notes

- *Differential tracking* - Only creates events on first run or when type `createdDate` changes
 - *One row per column* - Each column in each type generates one row
 - *System type exclusion* - Types in `System` or `Global` namespaces are excluded
 - *StriimWatcher type exclusion* - Types created by StriimWatcher itself are excluded
 - *App name matching* - Requires `IncludeAppDetail = true` to populate `appName` field
 - *Memory intensive* - Examines all types in the system; may impact performance in environments with many types
 - *Schema tracking* - Useful for tracking schema changes and type evolution over time
 - *Use cases*: Schema documentation, type inventory, schema change detection, data lineage, metadata management
-